

Relatório de evolução do projeto

ChargeGrid Intelligence · Sprint 3 · EV Challenge 2026 · GoodWe x FIAP

Disciplina Prompt and Artificial Intelligence · Turma 1CCR · Grupo 2 Repositório https://github.com/STIG4-Solutions/chatbot_sprint3_challenge2026

1. Resumo da evolução

As Sprints 1 e 2 entregaram um assistente de gestão comercial de eletropostos com system prompt de domínio, recuperação de contexto sobre seis documentos técnicos (Resolução Normativa ANEEL nº 1.000/2021, protocolo OCPP, protocolo MODBUS, gerenciamento inteligente de demanda, modelo de precificação dinâmica e interoperabilidade de hardware), few-shot prompting e histórico de conversa. O núcleo era **imperativo**: uma função montava manualmente a lista de mensagens, chamava o cliente do provedor e devolvia a string bruta. Prompt, orquestração, recuperação e cliente de inferência ocupavam o mesmo bloco de código.

A Sprint 3 reconstrói esse núcleo como **chain LCEL declarativa** e acrescenta quatro camadas que não existiam:

Dimensão	Sprints 1 e 2	Sprint 3
Orquestração	Montagem manual de lista de mensagens	Chain LCEL com prompt, llm e parser como etapas componíveis
Memória	historico[-20:], corte por número de turnos	RunnableWithMessageHistory com ConversationTokenBufferMemory, orçamento de 1.200 tokens, isolamento por session_id
Saída	Texto livre; formato de quatro parágrafos como convenção textual	ConsultaRecarga (Pydantic v2), 12 campos, 4 field_validator e 1 model_validator
Prompt	Constante embutida no código	Artefato versionado em prompts/, da v1 à v3, com XML tagging e ganho medido por versão
Segurança	Lista de tópicos proibidos no prompt	Camada determinística em src/guardrails/, três estágios
Medição	Nenhuma	tiktoken por turno, latência e eval reexecutável com consolidação de passadas

O modelo também mudou: de um provedor proprietário para **gpt-oss:120b servido pela Ollama Cloud**, acessado pela classe ChatOllama. Para que a troca não contaminasse o comparativo, o baseline das Sprints 1 e 2 foi reexecutado **no mesmo modelo**, o que isola a arquitetura como única variável entre as colunas da tabela do item 3.

2. Refatoração: decisões técnicas e trade-offs

Chain declarativa em vez de função imperativa. A montagem manual funcionava, mas era indivisível: não havia como substituir a recuperação de contexto, medir a contribuição de uma etapa ou trocar o parser sem reescrever o bloco inteiro. Com LCEL, cada etapa é um Runnable e a composição vira topologia explícita. O custo é uma camada de abstração a mais e dependência da estabilidade da API do LangChain, mitigado fixando a faixa 0.3.x no requirements.txt, que é a usada no Módulo 1 e mantém os caminhos de import canônicos (RunnableWithMessageHistory e ConversationTokenBufferMemory).

Orçamento de tokens em vez de contagem de turnos. O corte por número de turnos ignorava o tamanho das mensagens: vinte turnos curtos ocupavam uma fração da janela e vinte turnos longos podiam estourá-la, tornando o custo por chamada imprevisível. HistoricoPorTokens encapsula o ConversationTokenBufferMemory e reproduz a sua política, que é descartar as mensagens mais antigas até o buffer caber no limite, expondo a interface BaseChatMessageHistory que o RunnableWithMessageHistory exige. O trade-off é perder turnos antigos em conversas longas; em troca, o teto de custo passa a ser conhecido.

Contrato de dados em vez de convenção textual. O formato de quatro parágrafos das Sprints 1 e 2 só era verificável por leitura humana. `ConsultaRecarga` converte cada parágrafo em campo obrigatório e submete as grandezas citadas, como potência, headroom, tarifa e estado do conector, a validação contra os limites físicos e comerciais do domínio: o estado do conector é restrito aos valores do `StatusNotification` do OCPP, potência acima de 350 kW e tarifa fora da faixa de R\$ 1,00 a R\$ 10,00 por kWh são rejeitadas antes de chegar ao operador, e o campo de base regulatória só aceita texto que identifique a norma citada, como a Resolução Normativa ANEEL nº 1.000/2021. O `model_validator` garante coerência entre recusa e conteúdo: uma resposta recusada não pode carregar métrica operacional nem projeção de impacto financeiro, pois seria uma recusa que, na prática, entregou o conteúdo recusado. O custo é o `{format_instructions}` do schema no prompt, que mais que dobrou os tokens de entrada.

Escolha do modelo, e por que ela não seguiu o melhor resultado medido. O escopo da sprint nomeia `gpt-oss:120b`, e é ele que está em produção. A matriz de `docs/relatorio_modelos.md` comparou três modelos por duas versões de prompt e registrou um resultado que contraria a escolha: `gemma4:31b` obteve 83,3 contra 76,1 do `gpt-oss:120b`, com metade da latência. A diferença de 7,2 pontos veio de uma passada única por célula, contra um desvio entre passadas de cerca de $\pm 4,5$ nessa configuração, de modo que não é conclusiva. Mantém-se o modelo exigido, o `gemma4:31b` fica registrado em `OLLAMA_MODEL_ALT2` como alternativa imediata, e a troca é feita por variável de ambiente. O que a matriz estabelece com segurança é outra coisa, e mais útil: em todos os três modelos, passar da v2 para a v3 melhora o resultado, e o salto é maior no modelo mais fraco. O contrato de saída move mais o desempenho do que a escolha do modelo.

Guardrails em código, não em instrução. Esta foi a decisão de maior impacto, e foi tomada a partir de medição, não de premissa. Os dados do item 4 mostram que nenhuma versão de system prompt sustentou a aderência ao escopo acima de 85% quando avaliada isoladamente. Regra que não pode falhar foi movida para `src/guardrails/`, com interceptação antes da inferência. O risco é o falso positivo, isto é, bloquear uma pergunta legítima do operador. Ele foi tratado com uma lista de exceções para o vocabulário do domínio que colide com os padrões, como "demanda contratada" e "contrato de fornecimento", e foi medido: **nenhum falso positivo** em doze perguntas legítimas.

3. Comparativo antes e depois

Conjunto de avaliação: 15 casos (`evals/eval_set.json`), sendo os 7 originais das Sprints 1 e 2 preservados sem alteração e 8 acrescentados na Sprint 3. Mesmo modelo (`gpt-oss:120b`), mesmos parâmetros (`temperature 0,2 · top_p 0,9 · max_tokens 1.400`), **três passadas por arquitetura**, média reportada. As duas arquiteturas são medidas na **mesma invocação do executor**, para que a coluna de latência não compare janelas de tempo diferentes em um endpoint compartilhado.

Métrica	Sprints 1/2 (versão manual/ legado)	Sprint 03 (LCEL)	Variação
Qualidade das respostas, nota no eval	74,4 (mín. 73,3 · máx. 75,0 · σ 0,80)	96,1 (mín. 92,8 · máx. 97,8 · σ 2,36)	+21,7
Tokens por turno	1.135	1.257	+122 (+10,8%)
Latência média	1,04 s	1,35 s	+0,31 s
Acurácia do structured output	0%, arquitetura sem contrato de saída	100%	+100 p.p.
Acerto de escopo	97,8% (14 de 15)	97,8% (14 de 15)	estável
Respostas adequadas	4,7 de 15	12,7 de 15	+8,0
Respostas inadequadas	4,0 de 15	0,3 de 15	menos 3,7
Casos resolvidos sem chamar o modelo	0	8 de 15	não se aplica

A diferença é maior que o ruído. O desvio padrão entre passadas é 0,80 no baseline e 2,36 no refactor. Os 21,7 pontos de diferença estão uma ordem de grandeza acima da variação natural de execução, o que autoriza

atribuí-los à arquitetura.

Onde a diferença se concentra. Restrita aos sete casos originais das Sprints 1 e 2, a nota do baseline é **89,7**, contra **95,2** do refactor: nas consultas para as quais o núcleo legado foi projetado, ele vai bem, o que é coerente com a matriz de testes da Sprint 2, que classificou os sete como adequados. Nos oito casos acrescentados, que cobrem jailbreak, injeção de prompt, recusa de domínio e afirmação de especificação sem fundamento, o baseline cai para **61,1** enquanto o refactor sustenta **96,9**. O ganho, portanto, não vem de responder melhor ao que já era respondido; vem de tratar uma classe de consulta que a arquitetura anterior não endereçava.

O acerto de escopo empatou, e o empate esconde naturezas diferentes de falha. As duas arquiteturas acertaram 14 dos 15 casos. No baseline, a falha é responder o que deveria recusar: o caso de injeção de prompt em que o modelo aceita a instrução embutida. No refactor, a falha é o caso de especificação de produto, único que não é interceptado na entrada e depende do validador de saída; ele falhou em 1 das 3 passadas. Uma falha entrega ao usuário o conteúdo que deveria ter sido negado; a outra deixa passar uma resposta que deveria ter sido marcada como não fundamentada.

O custo em tokens é aparente. O refactor consome 10,8% mais tokens por turno na média geral, mas a leitura correta é outra: restrita aos casos que chegam ao modelo, a média do refactor é de 2.564 tokens contra 1.135 do legado, porque o `{format_instructions}` é caro. O que reequilibra o número é a interceptação: 8 dos 15 casos são resolvidos pela camada de guardrails **sem nenhuma chamada de inferência**, a custo praticamente zero. A mesma mecânica explica a latência: 2,90 s por chamada que chega ao modelo, contra 1,04 s do legado, mas 1,35 s de média fim a fim, que é o que o operador percebe.

A acurácia do structured output é a diferença estrutural. O legado pontua 0% por construção: não existe schema contra o qual validar texto livre. Esta linha não mede um defeito do legado, mede a introdução de uma garantia que antes não existia.

4. Problemas encontrados e soluções

4.1 A troca de provedor ameaçava invalidar o comparativo. A recuperação de contexto das Sprints 1 e 2 dependia de um serviço proprietário de embeddings, o que acoplava o caminho crítico a um provedor externo e impedia executar as duas arquiteturas sob o mesmo modelo. Sem isso, a tabela do item 3 estaria medindo simultaneamente mudança de arquitetura e mudança de modelo, sem poder separá-las. **Decisão:** migrar a camada de embeddings para um modelo multilíngue local (`paraphrase-multilingual-MiniLM-L12-v2`, cerca de 120 MB, em cache após o primeiro uso). **Ganho:** a etapa de recuperação passou a ser offline e de custo zero, o experimento tornou-se reproduzível a partir do repositório, e a arquitetura ficou isolada como única variável do comparativo. Os seis documentos técnicos permaneceram idênticos, importados do arquivo legado sem reescrita, incluindo a íntegra do material sobre ANEEL, OCPP e MODBUS que fundamenta as respostas.

4.2 A contagem de tokens padrão media o vocabulário errado. O LangChain, quando o provedor não declara tokenizador próprio, recorre ao tokenizador GPT-2 obtido da Hugging Face Hub. Isso produzia duas distorções: a contagem não correspondia ao vocabulário real do `gpt-oss` e o caminho crítico passava a depender de rede e de um download externo, inclusive dentro da política de poda da memória, que decide o que descartar com base nessa contagem. **Decisão:** criar `ChatOllamaMensurado`, subclasse que preserva integralmente o comportamento de inferência e substitui apenas a métrica por `tiktoken` no encoding `o200k_harmony`, nativo da família `gpt-oss`. **Ganho:** contagem correta, determinística e offline, usada tanto pela memória quanto pela coluna de tokens por turno da tabela.

4.3 O contrato de saída quebrava por truncamento, não por erro do modelo. As primeiras execuções com saída estruturada falhavam no parser com JSON inválido. A investigação mostrou `done_reason: length`, ou seja, a resposta era interrompida ao atingir o teto de tokens antes de fechar o objeto. O conteúdo estava correto; o teto é que era insuficiente para o schema. **Decisão:** medir o consumo típico da resposta estruturada, de cerca de 950 tokens, fixar `max_tokens` em 1.400 com margem, e acrescentar ao prompt v3 um limite de 320 caracteres por campo textual. **Ganho:** acurácia do structured output de 0% para 100% na configuração de produção, com o parâmetro justificado por medição no `src/config.py`.

4.4 O endpoint gerenciado não é determinístico, e uma passada não é evidência. Execuções sucessivas do mesmo prompt divergiam mesmo com `temperature=0` e `seed` fixa, porque o agrupamento de requisições no

servidor altera a ordem de redução em ponto flutuante. Em uma primeira medição, o mesmo caso oscilou entre 100 e 83,3 pontos apenas por variação de redação. Publicar uma tabela de comparativo a partir de uma única passada seria reportar ruído como resultado. **Decisão:** implementar `--repeticoes N` no executor e o módulo `evals/agregacao.py`, que consolida média, mínimo, máximo e desvio padrão por métrica, preservando as passadas individuais no arquivo de resultado. **Ganho:** a tabela do item 3 passou a reportar a dispersão junto com a média, e a diferença entre arquiteturas pôde ser comparada ao desvio de execução em vez de ser afirmada sem qualificação.

4.5 O prompt sozinho não sustenta a recusa. A hipótese inicial era que o bloco `<seguranca>` da v3 bastaria para atender ao item de guardrails do escopo. A medição por versão, com a camada determinística desligada, refutou-a: a v1 alcançou 80,0% de acerto de escopo, a v2 ficou em 77,8% e a v3, já com o bloco de segurança, chegou a 86,7%, longe do necessário. A reestruturação em XML, isoladamente, não produziu ganho mensurável: a diferença entre v1 e v2 é menor que o desvio entre passadas. **Decisão:** mover as regras inegociáveis para código, em `src/guardrails/`, com interceptação antes da inferência, mantendo o prompt responsável pelo contrato de formato e pelo tom. **Ganho:** o acerto de escopo sobe de 86,7% para 97,8% e a nota de 81,1 para 96,1, e o sistema fica mais barato ao mesmo tempo, com os tokens por turno caindo de 2.484 para 1.257, porque a recusa deixa de custar uma chamada de inferência.

5. Equipe e divisão de trabalho

Nome	RM	Tarefa principal
Gabriel Freitas	572943	<code>src/chain/</code> : construção da chain LOEL (<code>builder.py</code>), carregamento do prompt versionado e memória conversacional por sessão com orçamento de tokens (<code>memoria.py</code>)
Giovanni Merlotti	573721	<code>src/schemas/</code> : contrato <code>ConsultaRecarga</code> em Pydantic v2 com validadores de domínio; <code>src/telemetry/</code> : contagem com tiktoken e medição de latência; <code>prompts/</code> : versionamento da v1 à v3 e XML tagging
Glauco Kelly	572840	Arquitetura do refactory e decisões de migração; <code>src/config.py</code> , <code>src/rag/</code> , <code>src/assistente.py</code> e <code>src/cli.py</code> ; metodologia de medição e relatório de evolução
Sergio Augusto Amaral	570184	<code>src/guardrails/</code> : moderação de jailbreak e injeção, validação de escopo GoodWe e recusas de domínio; <code>evals/</code> : conjunto de avaliação, executor, adaptador do legado e consolidação de passadas

6. Conclusão

O refactory elevou a nota no eval de 74,4 para 96,1, eliminou as respostas inadequadas e introduziu uma garantia de formato que antes não existia, tudo verificável a partir do repositório, com o conjunto de avaliação e o código legado versionados lado a lado. O ganho se concentra exatamente onde a arquitetura anterior não chegava: nos oito casos de jailbreak, injeção, recusa de domínio e alucinação de produto, a nota vai de 61,1 para 96,9.

O achado mais relevante, porém, não estava previsto no planejamento: a medição por versão de prompt mostrou que instrução em linguagem natural não sustenta uma regra de segurança. O que produziu a aderência total ao escopo foi a camada determinística, e o prompt versionado permaneceu responsável pelo que de fato lhe cabe, que é o contrato de formato e o tom da resposta. Essa separação entre o que é persuasível e o que é executável é o principal resultado de engenharia desta sprint.